North South University

Department of Electrical & Computer Engineering

Project Report

Smart Library Management & Book System & Alert for the Dues (LMS)

Course Code: CSE327
Section: 06

Submitted by:

Gazi Shahriar Alam 2232629042
Imrul Kayes Uday 2231002042
Maisha Subat 2231161042

Submitted to:

Faculty: Mr. AKM Iqtidar Newaz [IqN]

Proposed Functional Requirements:

1. User Management: The system must allow administrators to register, update, and delete member and staff accounts.

2. Catalog Management: Librarians must be able to add, update, and remove library items (Books, Journals, DVDs).

3. Search and Filter: Users must be able to search for items by title, author, genre, or publication year.

4. Transaction Processing: The system must handle the checking out, returning, and renewing of items.

5. Hold/Reserve System: Patrons must be able to place a hold on an unavailable item and receive notifications when it is returned.

6. Fine Management: The system must automatically calculate overdue fines based on the user's membership type and the item's delay period.

Completed Functional Requirements:

1. User Management System:
Code:
UI:

2. Catalog Management:
Code:
UI:

3. Search & Filter: Not completed

4. Transaction Processing

Code:

UI:

5. Hold/Reserve System: Not completed

6. Fine Management:

Code:

UI:

Non-Functional Requirements:

1. Performance: The system should return search queries in under 2 seconds.

2. Security: Passwords must be encrypted, and unauthorized users must not be able to access staff-only portals.

3. Scalability: The system must support up to 10,000 concurrent users and a catalog of over 500,000 items.

4. Reliability: The system should have an uptime of 99.9%

Completed non-functional requirements:

1. Performance:

Code:

```
    @PostMapping("/pay-fine")
    public String payFine(@RequestParam Long itemId,
@RequestParam String method, HttpSession session) {
        Member dbUser =
memberRepository.findById(((Member)session.getAttribu
te("loggedInUser")).getId()).orElse(null);
        LibraryItemEntity item =
itemRepository.findById(itemId).orElse(null);

        if (item != null && dbUser != null) {
            long daysLate =
ChronoUnit.DAYS.between(item.getDueDate(),
LocalDate.now());
            double amount = 0.0;

            if (daysLate > 0) {
                FineCalculator calculator = new
FineCalculator();
                calculator.setStrategy(new
StandardFine());
                amount = calculator.calculate((int)
daysLate) * 10;
            }

            item.setDueDate(LocalDate.now());
            itemRepository.save(item);
            totalFinesCollected += amount;

            pushNotif(null, "Revenue", "৳" + amount +
" paid via " + method);

            final double finalAmount = amount;
            CompletableFuture.runAsync(() ->
emailNotifier.sendReceiptEmail(dbUser, "Fine: " +
item.getTitle(), finalAmount, method));
            return "Payment successful!
Receipt emailed.";
        }
        return "Error processing payment.";
    }
```

2. Security:

```
    @PostMapping("/return")
    public String returnItem(@RequestParam Long
itemId, HttpSession session) {
        Member user = (Member)
session.getAttribute("loggedInUser");
        LibraryItemEntity item =
itemRepository.findById(itemId).orElse(null);
        if (item != null && !item.isAvailable()) {
            if
(!"ADMIN".equalsIgnoreCase(user.getRole())) {
                long daysLate =
ChronoUnit.DAYS.between(item.getDueDate(),
LocalDate.now());
                if (daysLate > 0) return "BLOCKED:
Pay fine first.";
            }
            item.setAvailable(true);
            item.setBorrowerEmail(null);
            item.setDueDate(null);
            itemRepository.save(item);

            pushNotif(null, "Activity",
user.getName() + " returned a book.");
            return "Returned.";
        }
        return "Error returning book.";
    }
```

3. Scalability:

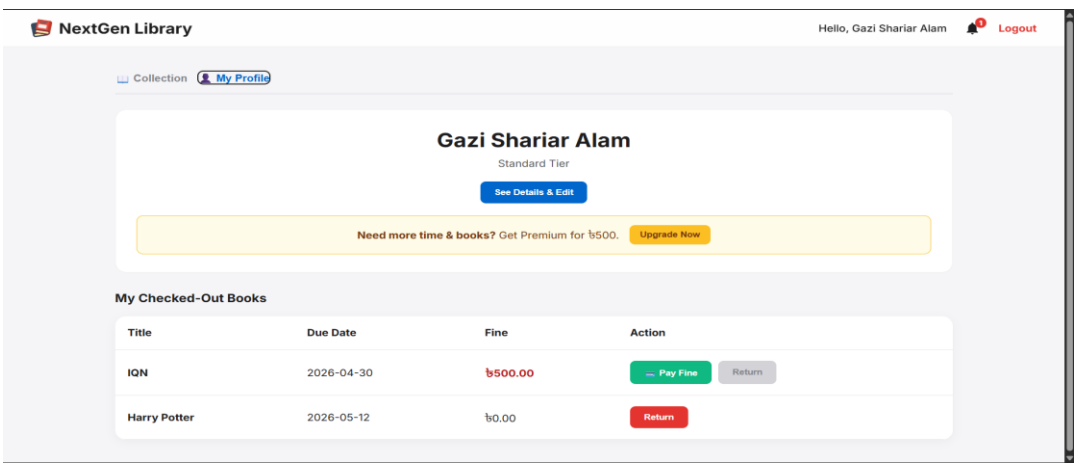
```
@RestController
@RequestMapping("/api")
public class LibraryApiController {

    @Autowired private LibraryItemRepository
itemRepository;
    @Autowired private MemberRepository
memberRepository;
    @Autowired private EmailNotifier emailNotifier;

    private static double totalFinesCollected = 0.0;
    private static List<AppNotification>
notifications = new CopyOnWriteArrayList<>();

    static class AppNotification {
        public long id = System.currentTimeMillis() +
(long)(Math.random()*1000);
        public String targetEmail; public String
title; public String message;
        public AppNotification(String t, String ti,
String m) { targetEmail=t; title=ti; message=m; }
    }

    private void pushNotif(String targetEmail, String
title, String message) {
        notifications.add(new
AppNotification(targetEmail, title, message));
        if(notifications.size() > 100)
notifications.remove(0);
    }
```

4. Reliability:

```
     @PostMapping("/pay-fine")
     public String payFine(@RequestParam Long itemId,
 @RequestParam String method, HttpSession session) {
         Member dbUser =
 memberRepository.findById(((Member)session.getAttribu
 te("loggedInUser")).getId()).orElse(null);
         LibraryItemEntity item =
 itemRepository.findById(itemId).orElse(null);

         if (item != null && dbUser != null) {
             long daysLate =
 ChronoUnit.DAYS.between(item.getDueDate(),
 LocalDate.now());
             double amount = 0.0;

             if (daysLate > 0) {
                 FineCalculator calculator = new
 FineCalculator();
                 calculator.setStrategy(new
 StandardFine());
                 amount = calculator.calculate((int)
 daysLate) * 10;
             }

             item.setDueDate(LocalDate.now());
             itemRepository.save(item);
             totalFinesCollected += amount;

             pushNotif(null, "Revenue", "৳" + amount +
 " paid via " + method);

             final double finalAmount = amount;
             CompletableFuture.runAsync(() ->
 emailNotifier.sendReceiptEmail(dbUser, "Fine: " +
 item.getTitle(), finalAmount, method));
             return "Payment successful!
 Receipt emailed.";
         }
         return "Error processing payment.";
     }
```

Strategy Design Pattern:

```java
@GetMapping("/inventory")
public List<LibraryItemEntity> getInventory() {
    List<LibraryItemEntity> items = itemRepository.findAll();

    // FIX: Splitting the strategy pattern into proper lines
    FineCalculator calculator = new FineCalculator();
    calculator.setStrategy(new StandardFine());

    for (LibraryItemEntity item : items) {
        if (!item.isAvailable() && item.getDueDate() != null) {
            long daysLate = ChronoUnit.DAYS.between(item.getDueDate(), LocalDate.now());
            if (daysLate > 0) {
                item.setCurrentFine(calculator.calculate((int) daysLate) * 10);
            } else {
                item.setCurrentFine(0.0);
            }
        }
    }
    return items;
}
```



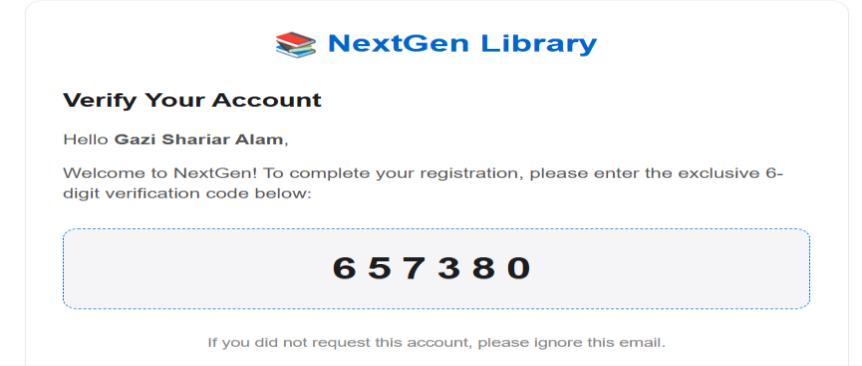
This design pattern is used to calculate fine algorithms. It has FineCalculator class that has different strategies. StandardFine strategy is instantiated and used to calculate the fine according to daysLate. This allows the library to swap logic without changing much inventory logic.

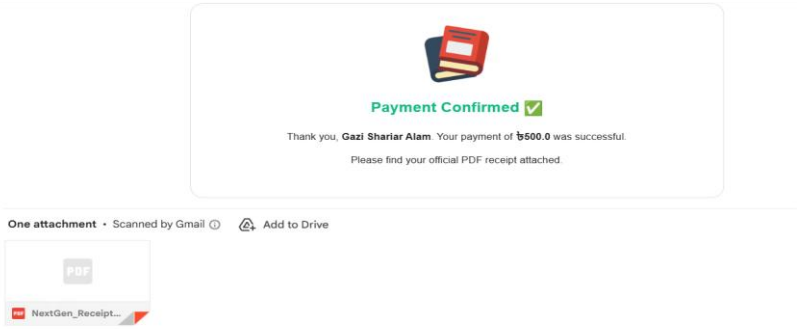
Observer Design Pattern

```java
@PostMapping("/register")
public String register(@RequestParam String name, @RequestParam String email, @RequestParam String password, @RequestPa
    if (!email.matches( regex: "^[A-Za-z0-9+_.-]+@(.+)$")) return "redirect:/register.html?error=invalid_email";
    if (memberRepository.findByEmail(email) != null) return "redirect:/register.html?error=exists";

    String code = String.format("%06d", new Random().nextInt( bound: 999999));
    Member newMember = new Member(name, email, password,  role: "STUDENT", university, code, mobile, gender, address, par
    memberRepository.save(newMember);

    CompletableFuture.runAsync(() -> emailNotifier.sendVerificationEmail(newMember));
    return "redirect:/verify.html?email=" + email;
}
```

```java
@Component
public class EmailNotifier {

    @Autowired private JavaMailSender mailSender;

    private final String logoUrl = "https://cdn-icons-png.flaticon.com/512/2232/2232688.png";  3 usages

    public void sendVerificationEmail(Member member) {  1 usage
        try {
            MimeMessage msg = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(msg,  multipart: true,  encoding: "UTF-8");
            helper.setTo(member.getEmail());
            helper.setSubject("Verify Your NextGen Account");

            String html = "<div style='font-family:Arial; max-width:600px; margin:auto; padding:30px; border-radius:16px;
                    + "<img src='" + logoUrl + "' width='80' style='margin-bottom:20px;'/>"
                    + "<h1 style='color:#0066cc;'>Welcome to NextGen!</h1>"
                    + "<p>Hello <b>" + member.getName() + "</b>, your 6-digit verification code is:</p>"
                    + "<div style='font-size:36px; font-weight:bold; letter-spacing:8px; background:#f5f5f7; padding:20px
            helper.setText(html,  html: true);
            mailSender.send(msg);
        } catch (Exception e) {}
    }
```

```java
public void sendReceiptEmail(Member member, String transactionName, double amount, String method) {  2 usages
    try {
        MimeMessage msg = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(msg,  multipart: true,  encoding: "UTF-8");
        helper.setTo(member.getEmail());
        helper.setSubject("Official Receipt - NextGen Library");

        String html = "<div style='font-family:Arial; max-width:600px; margin:auto; padding:30px; border-radius:16px;
                + "<img src='" + logoUrl + "' width='80'/>"
                + "<h2 style='color:#10b981;'>Payment Confirmed ✅</h2>"
                + "<p>Thank you, <b>" + member.getName() + "</b>. Your payment of <b>৳" + amount + "</b> was successfu
                + "<p>Please find your official PDF receipt attached.</p></div>";
        helper.setText(html,  html: true);
```





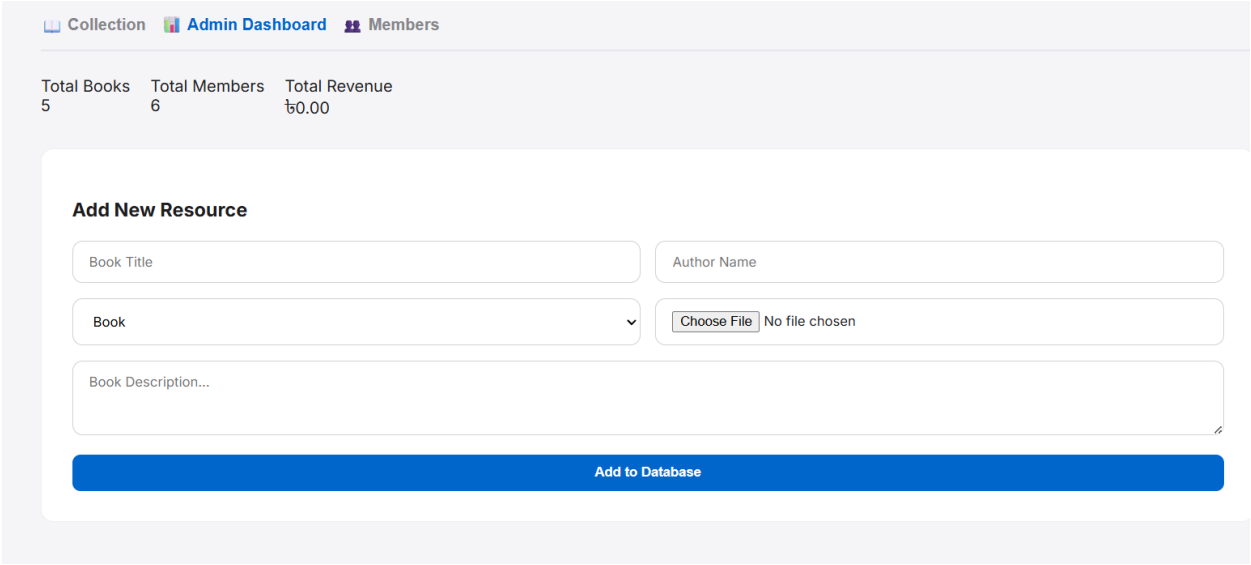
NEXTGEN LIBRARY RECEIPT

Transaction ID:	AC624BC0
Date:	2026-05-05
Billed To:	Gazi Shariar Alam
Item Description:	Premium Subscription
Payment Method:	bKash
TOTAL PAID:	**BDT 500.0**

This design pattern is used for notification systems in the application. While it mentions for “Book Availability,” the code demonstrates it through an email notification system. While registering, the system sends emailNotifier.sendVerificationEmail asynchronously. This decouples the registration logic from the notification delivery mechanism.

Factory Design Pattern

```java
public class ItemFactory {  no usages

    // The Factory Method
    public static LibraryItem createItem(String itemType, String title) {  no usages
        if (itemType == null || itemType.isEmpty()) {
            return null;
        }

        if (itemType.equalsIgnoreCase( anotherString: "BOOK")) {
            return new Book(title);
        }

        throw new IllegalArgumentException("Unknown item type: " + itemType);
    }
}
```

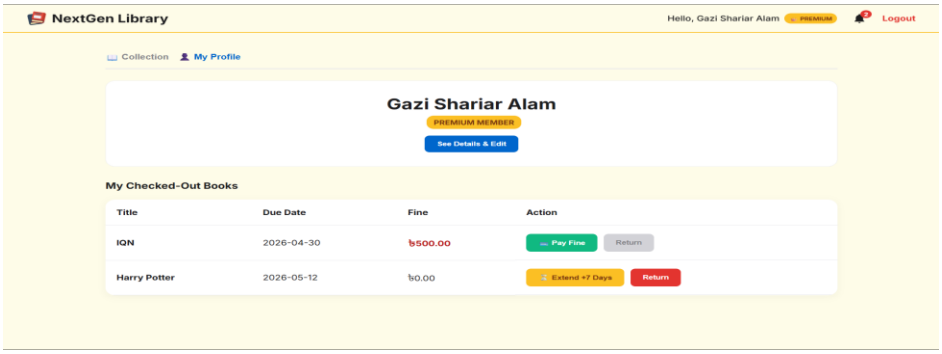


The library item creation is created by factory design pattern. It has ItemFactory class which includes createItem method and it takes an itemType string (e.g., "BOOK") and returns the corresponding object (e.g., new Book(title)). This encapsulates object creation, allowing the system to support new media types (DVDs, Journals) without modifying the admin dashboard logic.

Decorator Design Pattern (Premium Features)

```java
public abstract class AccountDecorator implements MemberAccount {  1 usage  1 inheritor
    protected MemberAccount decoratedAccount;  2 usages

    public AccountDecorator(MemberAccount decoratedAccount) {  1 usage
        this.decoratedAccount = decoratedAccount;
    }

    @Override  2 usages  1 override
    public int getBorrowLimit() {
        return decoratedAccount.getBorrowLimit();
    }
}
```

```java
@PostMapping("/extend-book")
public String extendBook(@RequestParam Long itemId, HttpSession session) {
    Member dbUser = memberRepository.findById(((Member)session.getAttribute( name: "loggedInUser")).getId()).orElse( othe
    LibraryItemEntity item = itemRepository.findById(itemId).orElse( other: null);
    if (item != null && dbUser != null && "PREMIUM".equals(dbUser.getRole())) {
        item.setDueDate(item.getDueDate().plusDays( daysToAdd: 7));
        itemRepository.save(item);
        return "Due date extended by 7 days!";
    }
    return "Error extending book.";
```

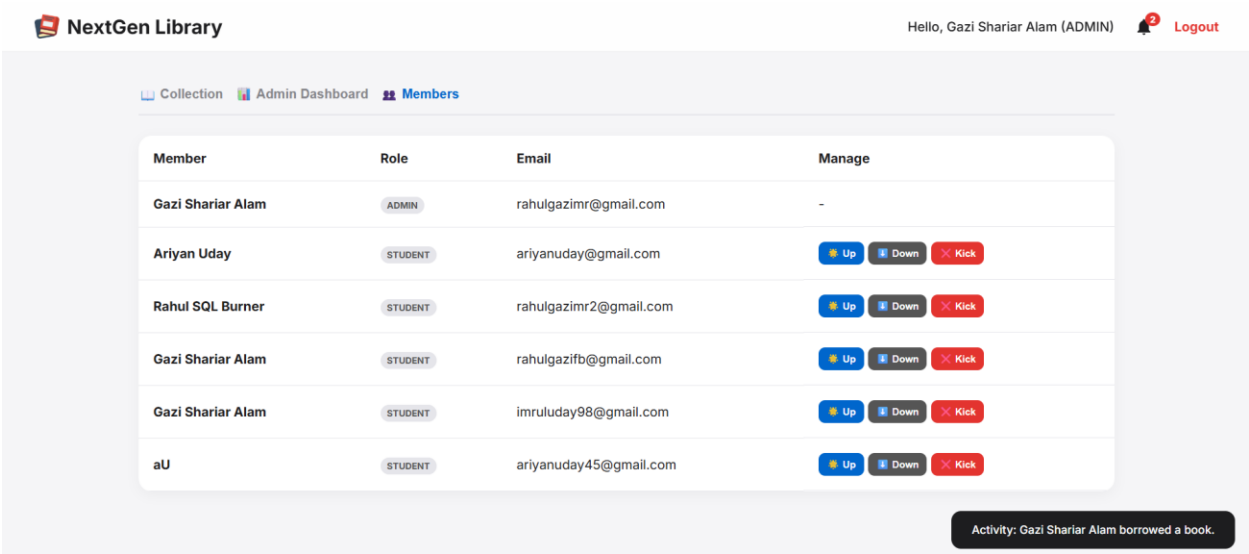


This design pattern is designed for premium account features. An AccountDecorator class implements the MemberAccount interface and wraps a decoratedAccount object. This is used to "wrap" standard users with **Premium** behaviors, such as the extendBook functionality, which allows specific members to add 7 days to their due date.

Repository Design Pattern (Database Access)

```java
@Repository  2 usages
public interface MemberRepository extends JpaRepository<Member, Long> {
    Member findByEmail(String email);  7 usages
}
```

```java
@PostMapping("/borrow")
public String borrowItem(@RequestParam Long itemId, HttpSession session) {
    Member user = (Member) session.getAttribute( name: "loggedInUser");
    LibraryItemEntity item = itemRepository.findById(itemId).orElse( other: null);
    if (item != null && item.isAvailable()) {
        item.setAvailable(false);
        item.setBorrowerEmail(user.getEmail());
        item.setDueDate(LocalDate.now().plusDays( daysToAdd: 7));
    //   item.setDueDate(LocalDate.now().minusDays(5));
        itemRepository.save(item);

        pushNotif( targetEmail: null,  title: "Activity",  message: user.getName() + " borrowed a book.");
        return "Borrowed: " + item.getTitle();
    }
    return "Error borrowing book.";
}
```



The system uses Spring Data JPA's Repository pattern. Interfaces like MemberRepository extend JpaRepository, abstracting complex SQL queries into simple method calls like findByEmail or save.

Singleton Design Pattern

```java
@RestController
@RequestMapping("/api")
public class LibraryApiController {

    @Autowired private LibraryItemRepository itemRepository;
    @Autowired private MemberRepository memberRepository;
    @Autowired private EmailNotifier emailNotifier;

    private static double totalFinesCollected = 0.0;  3 usages
    private static List<AppNotification> notifications = new CopyOnWriteArrayList<>();  4 usages

    static class AppNotification {  5 usages
        public long id = System.currentTimeMillis() + (long)(Math.random()*1000);  1 usage
        public String targetEmail; public String title; public String message;  3 usages
        public AppNotification(String t, String ti, String m) { targetEmail=t; title=ti; message=m; }  1 usage
    }

    private void pushNotif(String targetEmail, String title, String message) {  5 usages
        notifications.add(new AppNotification(targetEmail, title, message));
        if(notifications.size() > 100) notifications.remove( index: 0);
    }
```

This design pattern identifies for the DatabaseManager to ensure a single shared connection instance. Additionally, the LibraryApiController utilizes static shared members like totalFinesCollected and a notifications list to maintain global state across the application.

Diagram vs. Implementation Analysis

Class Diagram Match:

- **Strategy Pattern:** The diagram shows a FineStrategy interface with a StandardFine implementation. This matches the code exactly, where FineCalculator sets a strategy of type StandardFine.
- **Singleton Pattern:** The diagram lists DatabaseManager with a static instance and getInstance() method. This aligns with the architectural goal of preventing resource exhaustion by sharing a single connection.

- **Changes:** The Class Diagram illustrates a MediaCreator and BookCreator hierarchy for the Factory pattern. However, the actual implementation uses a simplified **Static Factory** (ItemFactory) with a switch-case/if-else block rather than a full class inheritance structure for the creators.

Sequence Diagram Match:

- **Borrowing Process:** The sequence diagram shows the Patron requesting a book through the System UI, which then communicates with an AuthProxy and InventoryMgr.
- **Changes:**
  - **Proxy Logic:** While the diagram depicts a separate AuthProxy object, the implementation shows role-based access checks (e.g., checking for "PREMIUM") directly within the Controller methods using the HttpSession.
  - **Inventory Management:** The "InventoryMgr" in the diagram is implemented via the itemRepository in the code. The code successfully follows the sequence of checking item availability, updating status to "Borrowed," and saving the transaction to the database.

GitHub Link:

https://github.com/imruluday/Smart-Library-Management-Book-System-Alert-for-the-Dues